

AI BASED HANDWRITING REPLICATION APP

HARIS JAVED
02-134221-010

ZARA ARIF
02-134221-100

HAMZA HAMEED
02-134221-102

A project report submitted in partial fulfilment of the

Requirements for the award of the degree of

Bachelor of Science in Computer Science

Department of Computer
Science Bahria University,
Karachi

2025

DECLARATION

We hereby declare that this project report is based on our original work except for citations and quotations which have been duly acknowledged. We also declare that it has not been previously and concurrently submitted for any other degree or award at Bahria University or other institutions.

Name : Zara Arif

Reg No. : 77244

Name : Haris Javed

Reg No. : 77173

Name : Hamza Hameed

Reg No. : 77246

APPROVAL FOR SUBMISSION

We certify that this project report entitled **"AI BASED HANDWRITING REPLICATION APP"** was prepared by **HARIS JAVED, ZARA ARIF & HAMZA HAMEED** has met the required standard for submission in partial fulfilment of the requirements for the award of Bachelor of Science in **Computer Science** at Bahria University, Karachi.

Approved by,

Supervisor : Miss Asia Samreen

The copyright of this report belongs to Bahria University according to the Intellectual Property Policy of Bahria University BUORIC-P15 amended on April 2019. Due acknowledgement shall always be made of the use of any material contained in, or derived from, this report.

© 2025 Bahria University. All right reserved.

I wholeheartedly dedicate this project to my family, whose endless love, prayers, and encouragement have been the true driving force behind every step of my journey. To my mother, whose sacrifices and unconditional support have shaped who I am today, and to my friends whose constant guidance and faith in me gave me the courage to keep moving forward. Their belief in my abilities has been my greatest motivation, and this achievement is as much theirs as it is mine.

(Zara Arif)

I lovingly dedicate this work to my family, whose love, prayers, and sacrifices have been my greatest strength. My mother's unwavering support, and my father's constant encouragement, gave me the courage to overcome challenges and the motivation to achieve this milestone. This accomplishment belongs to them as much as it does to me.

(Hamza Hameed)

I also dedicate this project to my mentors and friends, whose guidance, inspiration, and companionship have shaped my journey. My family, with their positivity and constant motivation, have made this journey easier, brighter, and more memorable. Together, they not only provided me with direction and confidence but also instilled in me the spirit to keep moving forward with determination.

(Haris Javed)

ACKNOWLEDGEMENTS

We would like to thank everyone who had contributed to the successful completion of this project. We would like to express my gratitude to my project supervisor, Miss Asia Samreen for her invaluable advice, guidance and her enormous patience throughout the development of the research.

In addition, we would also like to express our gratitude to our loving parents and friends who have helped and given us encouragement.

AI BASED HANDWRITING REPLICATION APP

ABSTRACT

This project presents the design and development of an artificial intelligence-based application capable of replicating an individual's handwriting style. Unlike traditional Optical Character Recognition systems that convert handwriting into plain text, this solution focuses on generating new content that mimics the unique handwriting style of the user. By combining advanced AI techniques with modern web development frameworks, the application offers a platform for personalized digital handwriting generation.

The system works by collecting handwriting samples uploaded by the user and applying preprocessing steps to prepare the data. The trained model then learns stroke patterns, spacing, and stylistic variations, enabling it to generate realistic handwritten text from typed input.

The application is built using FastAPI for the backend, React for the frontend, and MongoDB as the database. Key features include user authentication, secure document management, and export options such as PDF. Testing confirmed that the preprocessing pipeline and AI model significantly improved the authenticity and quality of the replicated handwriting.

This project highlights the potential of artificial intelligence in bridging the gap between traditional handwriting and digital communication. It emphasizes personalization, creative expression, and accessibility, while paving the way for future applications in education, design, and therapy.

TABLE OF CONTENTS

DECLARATION **ii**
APPROVAL FOR SUBMISSION **iii**
ACKNOWLEDGEMENTS **vi**
ABSTRACT **vii**
TABLE OF CONTENTS **viii**
LIST OF TABLES **x**
LIST OF FIGURES **xi**
LIST OF APPENDICES **xiii**

CHAPTER

1 INTRODUCTION **15**
1.1 Background 15
1.2 Problem Statement 17
1.3 Aims and Objectives 17
1.4 Scope of Project 18
1.4.1 Assumptions and Dependencies 18
1.4.2 Limitations 18
1.5 Chapter Summary 19

2 LITERATURE REVIEW **20**
2.1 Background 20
2.2 Related Work 21
2.2.1 Handwriting Recognition Systems 21
2.2.2 Handwriting Generation and Synthesis 22
2.3 Comparison Table with Existing Study 23
2.4 Chapter Summary 26

3 REQUIREMENT SPECIFICATIONS **27**
3.1 Existing Systems 27
3.2 Proposed System 27
3.3 Functional Requirements 28
3.4 Non-Functional Requirements 29
3.5 Use Cases 29

	3.5.1	Use Case Diagram	31
3.6	Chapter Summary		31
4	**DESIGN AND METHODOLOGY**		**32**
4.1	System Architecture		32
4.2	Technology Stack		33
4.3	Design of Key Modules		33
	4.3.1	Handwriting Detection Module	34
	4.3.2	Handwriting Generation Module	34
4.4	Chapter Summary		35
5	**SYSTEM IMPLEMENTATION**		**36**
5.1	Dataset Analysis		36
5.2	Data Preprocessing		36
5.3	Handwriting Detection Module		37
	5.3.1	Model Architecture	37
5.4	Handwriting Generation Module		38
	5.4.1	Model Architecture	38
5.5	System Integration		39
	5.5.1	Prerequisites	39
	5.5.2	Frontend Implementation	39
	5.5.3	Backend Implementation	43
	5.5.4	Backend API Call	44
5.6	Chapter Summary		46
6	**SYSTEM TESTING AND EVALUATION**		**47**
6.1	Testing Methodology		47
6.2	Functional Testing		47
	6.2.1	Landing Page Testing	47
	6.2.2	Login & Sign-Up Page Testing	48
	6.2.3	Handwriting Generator Page Testing	48
	6.2.4	Preview & Download Testing	48
6.3	Handwriting Detection Module Evaluation		49

6.4 Handwriting Generation Module Evaluation 49
6.5 System Usability & Performance Testing 50
6.5.1 Browser Testing 50
6.5.2 User Testing 52
6.5.3 Test Cases 53
6.6 API Testing 53
6.7 Chapter Summary 54

7 CONCLUSION 55
7.1 Key Takeaways 55
7.2 Limitations and Challenges 56
7.3 Future Developments 56
7.4 Chapter Summary 57

REFERENCES 55

LIST OF TABLES

TABLE	TITLE	PAGE
Table 2.1	Comparison of AI Models for Handwriting Recognition	21
Table 2.2	Comparison of Generative Models for Handwriting Generation	22
Table 3.1	Use Cases	26
Table 4.1	Technology Stack	30
Table 6.1	Test Cases	50

LIST OF FIGURES

FIGURE	TITLE	PAGE
Figure 1.1:	Workflow of handwritten text recognition system	14
Figure 1.2:	Handwriting Generation System	15
Figure 3.1:	Use Case Diagram	27
Figure 4.1:	System Architecture Diagram	29
Figure 4.1:	System Architecture Diagram	29
Figure 4.2:	Handwriting Detection Module Workflow	31
Figure 4.3:	Handwriting Generation Module Workflow	32
Figure 5.1:	Snippet from IAM Dataset	34
Figure 5.2:	InkWave Landing Page	37
Figure 5.3:	InkWave Sign Up Page	37
Figure 5.4:	InkWave Login Page	37
Figure 5.5:	InkWave Generate Word Page	38
Figure 5.6:	InkWave Generate Sentence Page	38
Figure 5.7:	InkWave Stylized Handwriting Generation Page	38
Figure 5.8:	Handwriting Detection Page	39
Figure 5.9:	Preview Output Page	39
Figure 5.10:	Custom Preprocessing of Handwriting Images for Compatibility with Model	40
Figure 5.11:	Resizing and Padding Handwriting Samples	41
Figure 5.12:	Handling Custom Style Features in Handwriting Generation	41
Figure 5.13:	Extraction Style Features	41
Figure 5.14:	Combining Multiple Handwriting Word Images into a Full Sentence Image	42

Figure 5.15: Main API POST Call 42

Figure 6.1: Web app compatibility on Chrome browser 48

Figure 6.2: Web app compatibility on Microsoft Edge browser 49

Figure 6.3: Web app compatibility on Opera browser 48

CHAPTER 1

INTRODUCTION

1.1 Background

The interaction with computers is fundamentally transformed by AI that has provided new opportunities of customization. Among the most interesting applications is the replication of handwriting which brings the traditional analog form of expression to the modern form of digital convenience. Handwriting is a way of expression of thoughts of a person, motor abilities and individual style that is why it is not just a method of sharing information, but it is the reflection of personal identity.

Researchers have over the years experimented with numerous methods of teaching computers how to create handwriting. Older models employed CNNs, RNNs and LSTMs to learn writing patterns and produce strokes at a time. These models were effective, yet the effect was frequently not as natural and random as in real handwriting. GANs became famous later to produce handwriting images directly. They would give more appealing results, however, they were hard to train and they did not always manage to maintain a steady style.

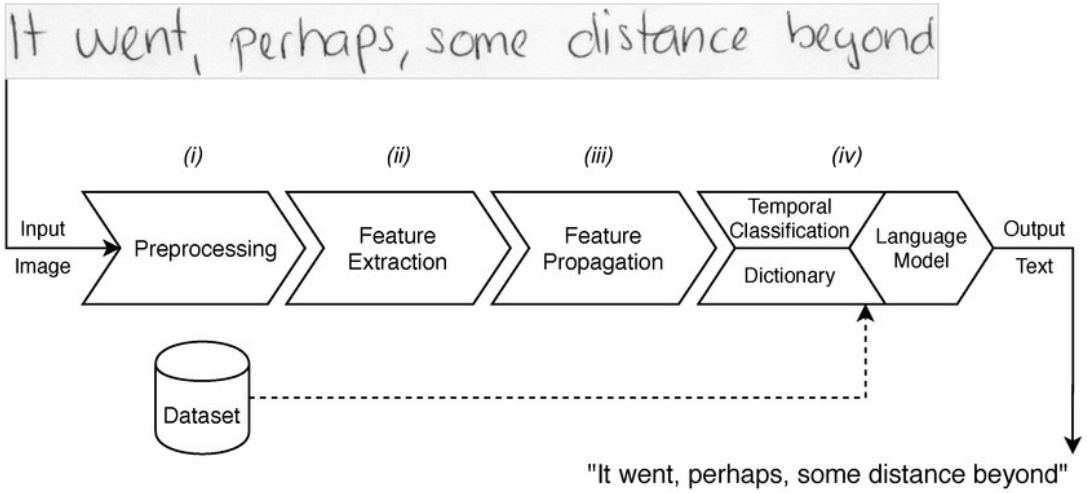


Figure 1.1: Workflow of handwritten text recognition system

With recent advances in generative AI, Latent Diffusion Models (LDMs) have become a much stronger option. Instead of trying to generate handwriting in one shot, diffusion models gradually refine noisy latent representations until a realistic

handwriting image forms. This makes it easier for them to capture small but important details that give handwriting a personality.

This project builds a user-friendly app that use a latent diffusion approach to learn a user's handwriting from a few sample images and generate new text in the same style. The goal is to make handwriting replication more accurate, more natural-looking, and easy for anyone to use, bridging the gap between traditional handwriting and modern digital tools.

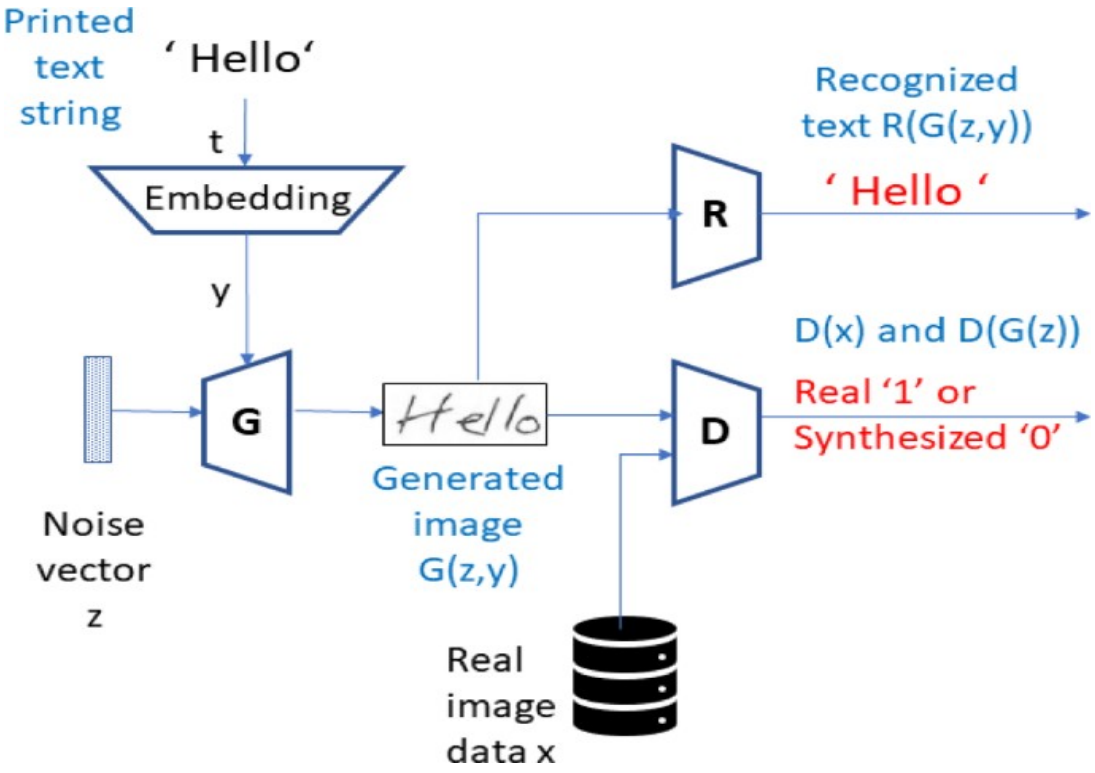


Figure 1.2: Handwriting Generation System

Beyond technology, it helps people with motor disabilities, such as dysgraphia, create readable handwriting effortlessly. Artists and designers can add personalized touches to their work, teachers can make materials more engaging, and it also has applications in document authentication, forensics, and digital preservation of historical manuscripts.

1.2 Problem Statement

Digital communication has resulted in the ease and speed of communication at the expense of personal expression. The Contemporary means are perfect in time, but they cannot express the individuality of handwriting that is personal and meaningful.

The existing OCR mechanisms effectively convert handwritten text into the digital one, but it deprives the style and uniqueness. They are also concerned with what is written rather than how it is written and thus not the right ones when one wants to create something using their own handwriting.

This gap affects many users. People who prefer to receive personalized handwritten details, teachers who appreciate handwritten documents, and those who maintain journals or personal mail, all find the anonymity of typed text to be a problem. People with such conditions as dysgraphia, tremors, or arthritis have even more problems as they cannot produce legible handwriting even being willing to do so.
Handwriting is complicated and depends on the pressure of the pen, speed, connections between the letters, spacing, and personal peculiarities. Any mechanism that emulates it needs to reproduce these subtleties in a manner that is consistent across the various texts. Most of the solutions available are substandard, they generate generic fonts, demand a lot of data, are difficult to operate, or are confined within their proprietary platform.

This project will address these issues by developing a convenient, precise and easy to use app to replicate handwriting. The system allows everyone to maintain the personal touch of their handwriting, even in the digital world through the advanced deep learning and careful designing.

1.3 Aims and Objectives

- To design and develop an AI-based handwriting replication system capable of learning an individual's handwriting style and generating text in the same style.
- Develop a responsive, user-friendly interface allowing users to upload handwriting samples, input custom text, preview results, and download generated handwriting.
- Provide output rendering with export options such as PNG images and PDF files while maintaining stroke consistency and overall style.
- Prepare comprehensive documentation covering system design, model architecture, preprocessing methods, training procedures, and deployment steps.

1.4 Scope of Project

This project will be executed with the scope of end-to-end development of an application that uses AI to replicate handwriting, including the design of the concept, the implementation of the model, interface development, and testing. The system is based on the idea of learning the handwriting of the user by being fed samples of the handwriting and producing new text visually similar to their natural handwriting. The project will consist of a full web-based application that will be developed with FastAPI as a backend, React as a user interface, and MongoDB as a secure data storage. The image preprocessing, handwriting style extraction, model inference and handwriting generation are implemented fully. One can upload samples of their handwriting, create text using their handwriting, preview handwriting, change settings and can export the generated outputs in the form of images or PDF files.

1.4.1 Assumptions and Dependencies

- Handwriting samples are clear and sufficient for style extraction.
- Reliable internet connectivity for users.
- Devices (laptops, tablets, or smartphones) to upload handwriting samples.
- Frontend is accessed via compatible browsers.

1.4.2 Limitations

- Replication accuracy may vary with sample quality or complexity.
- Supports English handwriting only.
- Highly cursive or messy handwriting may reduce fidelity.
- Long paragraphs may show slight spacing or alignment variations.

1.5 Chapter Summary

In this chapter, the author describes an AI based handwriting imitation system that can learn the unique writing style of a user and produce new text using that style. It emphasizes the inability of digital text to maintain the personal expression and demonstrates that deep learning models can correctly reproduce the handwriting patterns. The project will also seek to develop a scalable, accurate, and user-friendly system, including pre-processing, web-based deployment and model training. The

system is demarcated by the scope, assumptions and limitations, which are clear on what is provided and under which conditions it will be operated.

CHAPTER 2

LITERATURE REVIEW

2.1 Background

The handwriting recognition technologies have become highly necessary in digitizing documents and automation of text processing in a number of industries. Handwritten Text Recognition (HTR) systems have 95-98% accuracy on standard datasets such as MNIST and have been extensively deployed in document segmentation, form recognition systems, and authentication systems [10]. More recent studies have been making gains in recognition accuracy with generative AI post-processing, and Character Recognition Rates have been reported to be rising to 93.6 and later 94.9 in medical prescriptions [9]. Nevertheless, such systems are more oriented on text content extraction and do not provide an opportunity to save personal writing styles, and there is still a niche to be left to people who want to retain their own handwriting in the digital world.

The technique of handwriting synthesis has received interest due to its capacity to copyright styles of individuals to be used in education, digital marketing, forensic, and accessibility applications [12]. It has been found that handwriting has cognitive advantages compared to typing because it stimulates the neural pathways that lead to learning and memory formation. Recent developments in deep learning have offered several methods of handwriting generation. Research by GAN et al [1] showed that GANs can be used to generate realistic handwritten characters and that Gan et al. [1] created HiGAN to imitate handwriting using disentangled style representations.

LSTM-based systems have demonstrated sequential generation of handwriting and diffusion-based models have shown remarkable performance with Character Error Rates as low as 6.94% with only five samples of reference [2]. Luo et al. [14] also presented CalliRewrite, an unsupervised model of restoring handwriting behaviors, and robotic systems designed by Huang and Xiong [15] can be shown in practical applications in physical handwriting replication.

Although these technologies have this kind of advantage, the solutions currently available need specialized hardware to capture in real-time or they produce static fonts that are not authentic. The given project creates an AI-based application, which integrates the latest methods of deep learning with a usable interface and enables users to provide a sample of their handwriting and create new texts that would recreate their own writing style.

2.2 Related Work

This section reviews major research contributions in handwriting recognition and generation, focusing on recognition systems, synthesis approaches, and computational considerations.

2.2.1 Handwriting Recognition Systems

Deep learning has developed handwriting recognition. Previous HTR systems relied on handcrafted characteristics and domain expertise, thus complicating the OCR training [11]. Neural architecture has been enhanced in modern times which has enhanced its accuracy and generalization.

The accuracy of CNNs on MNIST was 99.87% [10], which is explained by the fact that preprocessing methods such as grayscale conversion, normalization, and augmentation are essential to think. Their study depicted that deep neural models are better than traditional shallow architectures.

J. Prem Kumar [16] experimented with a hybrid CNN-LSTM model, which reaches a CER of 4.57% and WER 12.3% on IAM using CNN spatial feature extraction with LSTM sequential model and CTC loss to align. This model was better than conventional OCR particularly when it came to mixed-script and cursive handwriting.

There are also the domain-specific systems. Yakovchuk and Vasin [9] enhanced medical prescription recognition with generative AI to do post-processing and raised Character Recognition Rate to 93.6-94.9% and Word Recognition Rate to 81.4-82.3, which highlighted the necessity of context-specific corrections.

Wang et al. [17] proposed a freeform handwriting authentication system that is based on energy-oriented self-supervised learning to perform authentication. Their

approach was found to have Top-1 accuracy of 60.43%-83.29% and Top-5 accuracy of 76.74%-96.10% on datasets (IAM, CEDAR, CVL, QUWI) and is thus found to be robust in real-world situations.

2.2.2 Handwriting Generation and Synthesis

The generation of handwriting has increased in great numbers, progressing beyond the old methods to deep generative models. The survey by Diaz et al. (2025) of synthesis strategies is between 2019 and 2024 where deep learning was predominant.

Systems based on GAN showed great potential. Different types of GAN (vanilla, conditional, Wasserstein) and their capability to generate a variety of handwriting are shown, although they are not always consistent in long sequences and control of fine details.

Gan et al. [1] proposed HiGAN, where CER = 12.54% and WER = 28.91%, and the style representations are disentangled. HiGAN+ also enhanced one-shot transfer of styles with writer-specific auxiliary and contextual loss functions.

Kang et al. [18] created GAN writing, a content-conditioned model that incorporates a recognition network trained using CTC loss. It produces the readable and styled handwriting, although the best results are achieved with several references samples.

State-of-the-art is currently in diffusion approaches. Nikolaidou et al. [2] suggested DiffusionPen, a few-shot diffusion model that learns handwriting style using only five samples, with CER 6.94% and WER 18.11. Its style extraction that is based on metric learning is effective in capturing familiar and unseen styles.

Dai et al. [19] created One-DM, a one-shot diffusion model of generating handwriting in the style of calligraphy. It can be used to imitate a realistic style using a single example by incorporating high-frequency components in the style module.

Generators that are based on LSTMs are concerned with synthesis on a stroke level, and their generation is fast enough to be used in real-time. The smooth trajectories of these models are less controllable than GANs and diffusion models.

2.3 Comparison Table with Existing Study

This section presents a comparative analysis of existing handwriting recognition and generation studies. The tables summarize objectives, methodologies, datasets, performance metrics, and key findings, highlighting the strengths and limitations of different models. This comparison provides a clear overview of the current research landscape and sets the context for the proposed work.

Table 2.1: Comparison of AI Models for Handwriting Recognition (Medical, SSL, RL)

Aspect	**Medical Prescriptions**	**Self-Supervised Learning**	**Relaxation Labelling**
Title	Increasing Accuracy of Handwriting Text Recognition in Medical Prescriptions with Generative AI [9]	Image-based Freeform Handwriting Authentication with Energy-oriented Self-Supervised Learning [17]	Boosting CNN-based Handwriting Recognition Systems with Learnable Relaxation Labelling [7]
Authors	Oleg Yakovchuk, Maksym Vasin	Jingyao Wang, Luntian Mou, Changwen Zheng, Wen Gao	Sara Ferro, Alessandro Torcinovich, Arianna Traviglia, Marcello Pelillo
Objective	Improve handwriting recognition for medical prescriptions using Generative AI	Enhance handwriting authentication using self-supervised learning	Improve CNN-based handwriting recognition with Relaxation Labelling
Problems Addressed	Low accuracy in recognizing messy, inconsistent handwriting in medical prescriptions	Difficulty in authenticating free form handwriting, especially under noise, forgery, and style variation	Limited generalization and inconsistency in CNN-based handwriting recognition models
Key Methodology	Used multiple handwriting recognition engines (Amazon, Google, Microsoft). Applied Generative AI for context-aware post-processing corrections	Contrastive and generative self-supervised learning Energy-oriented operator for noise filtering during feature extraction	Relaxation Labelling (RL) with CRNN and FCN Sparsification technique to accelerate convergence
Datasets Used	40 medical prescription images	IAM, CEDAR, CVL, QUWI, ICDAR2013, EN-HA	Saint Gall, Parzival, Washington, IAM
Evaluation Metrics	Character Recognition Rate (CRR) Word Recognition Rate (WRR)	Authentication accuracy (Top-1, Top-5) - Robustness to noise and forgery	Character Error Rate (CER) - Word Error Rate (WER)

Accuracy / Performance	Without AI: CRR 0.936, WRR 0.814 With AI (context-aware edits): CRR 0.949, WRR 0.823	Best (IAM-FHA): 83.29% Top-1, 96.10% Top-5 Worst (QUWI-FHA): 60.43% Top-1, 76.74% Top-5	CER improvements: IAM (5.40% → 5.33%), WER improvements: IAM (17.85%→16.67%)
Key Findings	AI improves recognition but may degrade results if applied blindly. Context-aware AI enhances accuracy	Self-supervised learning improves handwriting authentication. Noise reduction improves model robustness	RL improves recognition consistency and generalization. Competes with transformer models

Table 2.2: Comparison of Generative Models for Handwriting Generation (GAN, LSTM, Diffusion)

Aspect	**HiGAN (Handwriting Imitation GAN)**	**LSTM-Based Handwriting Generation**	**Diffusion-Based Handwriting Generation (DiffusionPen / LDM)**
Title	HiGAN: Handwriting Imitation Conditioned on Arbitrary-Length Texts and Disentangled Styles [1]	Generating and Smoothing Handwriting with Long Short-Term Memory Networks [3]	DiffusionPen: Few-Shot Diffusion Model for Handwriting Style Generation [2]
Model	Generative Adversarial Network (GAN)	Long Short-Term Memory (LSTM)	Latent Diffusion Model (LDM)
Objective	Generate variable-length handwriting while accurately imitating styles from reference samples	Generate and smooth handwriting sequences using LSTM networks	Generate and smooth handwriting sequences using LSTM networks
Methodology	GAN-based approach with style disentanglement; HiGAN+ uses writer-specific auxiliary and contextual loss	Sequential LSTM trained on stroke data; focuses on smooth generation	Latent diffusion models with style conditioning; few-shot style learning
Problems Addressed	Difficulty in transferring handwriting styles; challenges in variable-length sequence generation	Weak style control in sequential handwriting generation; smoothing needed for real-time applications	Limited number of reference samples; need for both short- and long-text generation with style fidelity
Style Imitation	Can imitate handwriting styles; requires multiple reference samples for optimal performance	Weak style control; primarily smooths handwriting rather than replicating style	Highly flexible style replication; can learn from as few as 5 reference samples
Datasets Used	IAM, CVL handwriting datasets	IAM, CVL handwriting datasets	IAM, GNHK datasets
Evaluation Metrics	Frechet Inception Distance (FID), Character Error Rate (CER), Word Error Rate (WER)	Subjective smoothness evaluation; LSTM loss metrics	CER, WER, Learned Perceptual Image Patch Similarity (LPIPS)

Accuracy / Performance	CER: 12.54%, WER: 28.91% (moderate performance)	No explicit recognition accuracy; focuses on handwriting smoothness	CER: 6.94%, WER: 18.11% (best performance, close to real handwriting)
Key Findings	Can generate realistic handwriting with style imitation; performance improves with multiple reference samples	LSTM effectively smooths handwriting; limited style control; good for real-time generation	Diffusion models achieve high-quality, flexible handwriting generation; few-shot capability enables efficient style replication

2.4 Chapter Summary

The chapter has examined prominent studies on handwriting generation, particularly on GAN, LSTM, and diffusion-based methods. HiGAN proved to be effective in the style imitation of the variable length sequences with multiple references. The LSTM-based models could give smooth handwriting generation but did not have control over the style. Neural diffusion models such as LDM were able to perform well on few-shot learning and model replicate style. Quality was usually measured using evaluation metrics like CER, WER, and FID. Generally, the literature shows a trend in the development of generative deep learning models that can be used to generate realistic and stylistically correct handwriting and overcome such issues as the lack of reference data and variability in sequence.

CHAPTER 3

REQUIREMENT SPECIFICATIONS

3.1 Existing Systems

Many applications related to handwriting are concentrated on either recognition or font creation, yet very few of them are based on the replication of real handwriting and real-world functionality. Current OCR systems translate the text written by hand into digital form but lose stylistic information. Font generation services generate computer fonts based on handwriting samples, which generate unnatural results where each character is the same.

The latest scholarly prototypes have shown amazing handwriting synthesis abilities. DiffusionPen by Nikolaidou et al. (2024) with latent diffusion models reached Character Error Rate of 6.94% with only five reference samples, whereas style imitation was demonstrated by the use of disentangled representations with HiGAN by Gan et al. (2022). Nevertheless, their existence is mainly in research usage and form of implementation that needs technical skills and cannot be easily used by the non-technical population. The gap is high among the users who desire to create new handwritten content using typed text without losing their own writing character.

3.2 Proposed System

This is a web-based application, which combines the use of handwriting sample, AI-based style learning, and real handwriting generation. The key features include:

- **Handwriting Sample Collection:** It allows users to submit 5-10 handwriting samples in an easy-to-use interface. The system accepts PDF, PNG and JPG.
- **AI-Powered Style Learning:** This system utilizes diffusion models to extract features. Preprocessing involves normalization, noise elimination and correction of alignment to ensure uniformity in input quality.
- **Handwriting Generation:** The user keys in the text and the system creates handwritten text that mimics his or her own handwriting.
- **Export Options:** It has several export options such as PNG images and PDF documents to print.
- **Easy to use Interface:** Responsive design that operates on the desktop and mobile platform with visual progress indicators.

- **Secure User Management:** Secure authentication of user accounts to enable the users to have full control of their handwriting data.

3.3 Functional Requirements

This section outlines the functional requirements of the proposed system.

FR1: The system shall allow users to create accounts and authenticate securely using email and password.
FR2: Users shall upload handwriting samples as images (eg PNG).
FR3: The system shall extract text from the uploaded handwriting sample using a handwriting recognition module and display it to the user.
FR4: Users shall be able to enter new text through a dedicated text input interface for handwriting generation.
FR5: Users shall also be able to generate handwriting by entering new text.
FR6: The system shall train or adapt a personalized handwriting generation model using user-provided handwriting samples.
FR7: The system shall generate handwriting output that replicates the user's handwriting style for the user-entered text.
FR8: Users shall be able to preview the generated handwriting before downloading or saving.
FR9: Users shall export the generated handwriting outputs in PNG and PDF formats.

3.4 Non-Functional Requirements

This section outlines the non-functional requirements of the proposed system.

- **Maintainability:** The code will follow design principles and standard code practice which is easy to maintain and update in future iterations.
- **Usability:** A user-friendly interface will be adopted, with a clean and simple design to make it easier for users to navigate and upload images.
- **Compatibility:** The system shall work on modern browsers.
- **Security**: User data shall be encrypted with industry-standard protocols. Users shall have complete control over their data.

3.5 Use Cases

This section outlines use cases of the proposed system.

Table 3.1: Use Cases

ID	Use Case	Actor	System Response	Outcome
1	Access Landing Page	User	System displays landing page and "Get Started" button	User understands the app's purpose and capabilities.
2	Navigate to Sign Up	User	System displays sign up form with fields for Full name, Email, Password, and creator terms checkbox	User can create a new account
3	Create Account	User	System validates input fields, creates encrypted account, stores credentials securely	User account successfully created and can access app features.
4	User Login	User	System authenticates credentials, validates user, redirects to handwriting generation dashboard	User gains access to handwriting generation features
5	Enter Custom Text	User	System provides text input area.	Text is ready for conversion
6	Upload Handwriting Samples	User	System displays upload area with a message Accepts image uploads	Handwriting samples uploaded for model training
7	Convert to Handwriting	User	System processes uploaded samples, trains/adapts model, generates handwriting in user's style, displays "Convert to handwriting" button with progress indication	Handwriting generation initiated
8	Preview Generated Handwriting	User	System displays generated handwriting image in preview panel.	User can view generated handwriting before downloading
9	Download as Image	User	System converts handwriting to image format (PNG),	Image file downloaded to user's device

			initiates download.	
10	Download as PDF	User	System converts handwriting to PDF format with proper formatting, initiates download	PDF file downloaded to user's device
11	Return to Dashboard	User	System navigates back to main generation page.	User returns to main interface
12	Sign Out	User	System terminates session, clears authentication, redirects to landing/login page	User logged out securely.

3.5.1 Use Case Diagram

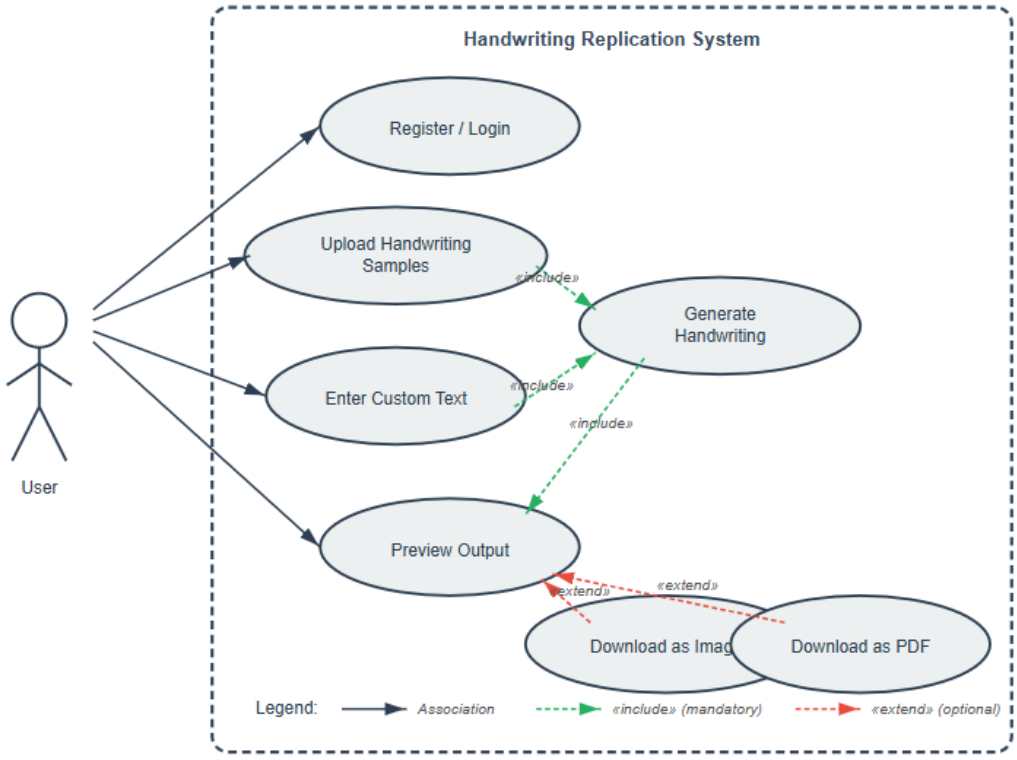


Figure 3.1: Use Case Diagram

3.6 Chapter Summary

This chapter presented the specifications of the requirement of the AI-Based Handwriting Replication Application. The discussion showed that current handwriting synthesis applications had deficiencies. The presented system will fill in these

loopholes by creating an integrated web application that incorporates sample collection, AI-controlled style learning, and genuine handwriting generation. The chapter addresses the functional and non-functional requirements and use cases of the system which show how the system can be used by the user.

CHAPTER 4

DESIGN AND METHODOLOGY

4.1 System Architecture

The AI-Based Handwriting Replication App has a straightforward and easy system architecture since users can upload handwriting samples, extract their writing style, and create new handwritten text. This section describes how the system works to process the samples, learn the handwriting of the user and how it can write realistically in handwritten form.

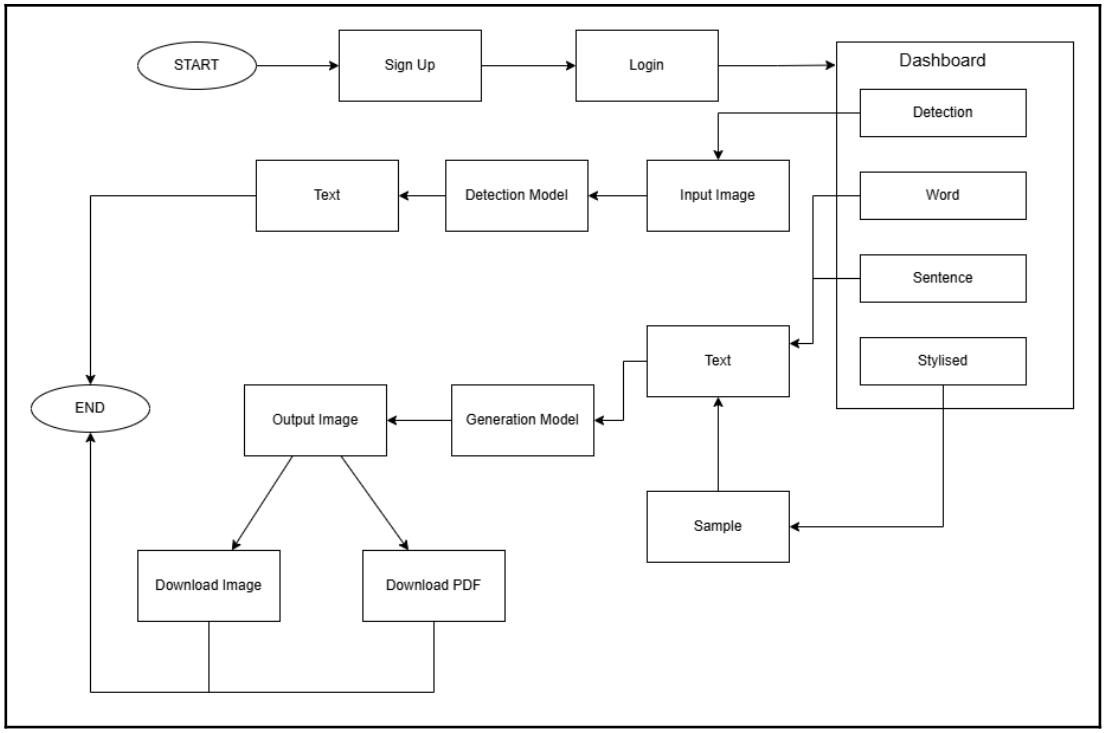


Figure 4.1: System Architecture Diagram

1. The system starts when the user posts 5 or more pictures of his or her handwriting samples to the application interface.
2. The user types in personal text within the text input interface which is the content that will be typed in the style of the user.
3. The samples of handwriting and typed text are sent to the Handwriting Generation Model, which is built on the diffusion based.
4. Architecture. The model combines handwriting to achieve a realistic handwritten image, which is similar to the style of the user.

5. The resultant output of the handwriting is transmitted back to the backend and converted into final formats, e.g. PNG or PDF, with correct resolution, padding and layout formatting.
6. The generated handwriting is also shown in the UI of the application, and before the user can download the output, he can preview it.
7. Lastly, the user can export, or download replicated handwriting as an image or PDF, and this completes the end-to-end handwriting replication process.

4.2 Technology Stack

The technology stack used is as follows:

Table 4.1: Technology Stack

Component	Technology Used	Description
Frontend	React.js	Provides the UI for sample upload, text input, preview, and downloads.
Backend	FastAPI	Handles API requests, preprocessing, model inference, and communication between UI and AI modules.
UI	Figma	Initial wireframes and prototypes were designed.
Database	MongoDB	Stores user accounts, handwriting samples, generated outputs, and related data.
Handwriting Detection Model	Microsoft TrOCR	Extracts text from the user's uploaded handwriting samples.
Handwriting Generation Model	Latent Diffusion Model	Learns the user's handwriting style and generates new handwritten text that matches their writing.
Hosting	Local	Hosts the application

4.3 Design of Key Modules

The proposed handwriting replication system is built using two core modules that work together: the Handwriting Detection Module and the Handwriting Generation Module. The following section explains the purpose and working of each component in detail.

4.3.1 Handwriting Detection Module

The role of this module is to detect and extract text in handwriting of the uploaded samples of the user. It is applied with a Tesseract based OCR model. The module makes sure that the samples uploaded have valid handwritten material and helps in checking the quality of the material before it is utilized in learning the style.

Handwriting detection starts by transforming the uploaded image into the machine-reading format. Preprocessing the image is done by cleaning the image with noise removal, resizing, and alignment correction. The OCR model is used to recognize the handwritten regions and remove the text after the sample is readied. This assists the system to affirm the existence of readable hand scripts.

The following block diagram illustrates the workflow of the handwriting detection module:

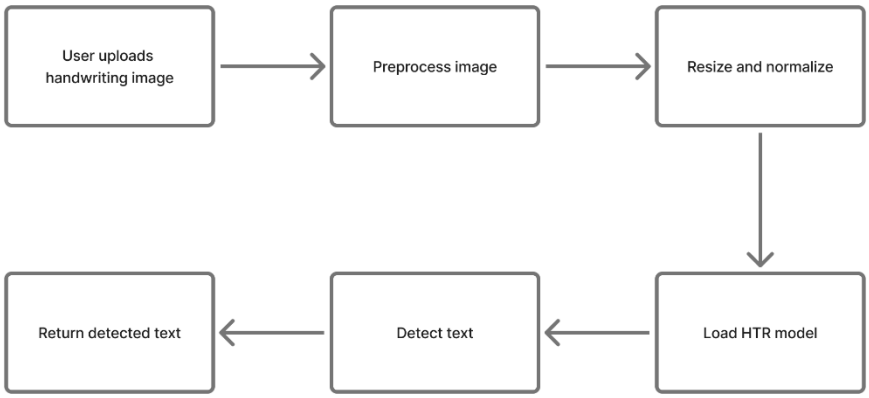


Figure 4.2: Handwriting Detection Module Workflow

4.3.2 Handwriting Generation Module

This module is the center of the system. It is based on replicating the personal handwriting of the user with the help of the Latent Diffusion Model (LDM). Once the system has checked the uploaded samples, the model will analyze the writing style

through learning the stroke patterns, character shapes, spacing, and stylistic variations. These attributes are coded into a style embedding which is a representation of the handwriting of a user.

After the user enters personal text, the generation module applies to the LDM to generate new handwriting which resembles extracted style. This model operates by successively decreasing latent noise to a realistic handwritten image. The resulting output is translated into a clear hand-written image which is a close replica of the natural writing of the user.

A block diagram showing the working of the handwriting generation module is given below:

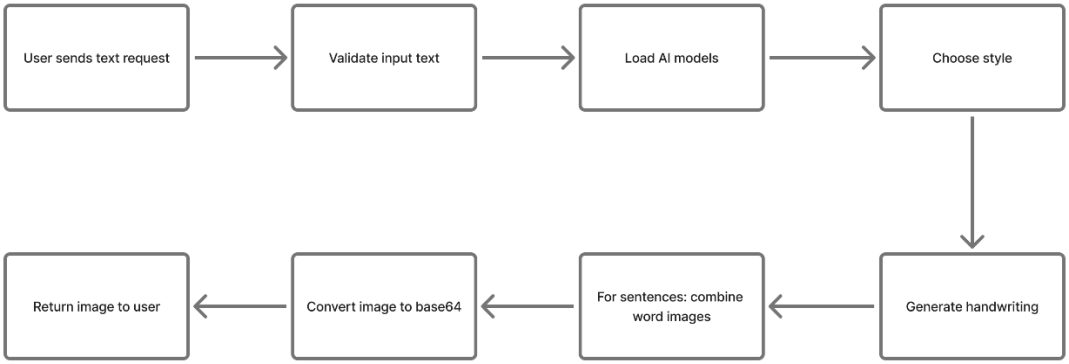


Figure 4.3: Handwriting Generation Module Workflow

4.4 Chapter Summary

This chapter described the design of the handwriting replication system with reference to the two key modules. The Handwriting Detection Module incorporates an OCR based on Tesseract to detect and remove handwritten samples to ensure that the system accepts clear and valid input. The Handwriting Generation Module is a Latent Diffusion Model (LDM) that (after being trained on the individual style of a user) can create realistic handwritten text of any user-provided text. This chapter gives a clear description of the input processing and output production making the basis of the implementation and evaluation of the system in the following chapters.

CHAPTER 5

SYSTEM IMPLEMENTATION

5.1 Dataset Analysis

InkWave utilizes the IAM Handwriting Dataset, which is a highly regarded resource in the field of handwriting generation and recognition. The IAM dataset offers a comprehensive collection of thousands of authentic handwritten samples from a diverse group of contributors, making it a reliable benchmark for training and evaluating handwriting models. It encompasses handwritten words, sentences, and entire paragraphs, each meticulously scanned from real documents. Additionally, the dataset provides segmented images at both line and word levels, writer identities to facilitate learning of individual writing styles, and ground-truth text for accurate mapping between written content and its visual representation. With over 1,500 handwritten pages and approximately 115,000 distinct words produced by 657 writers, the IAM dataset exposes models to a wide range of handwriting styles, stroke formations, slants, and stylistic variations. This diversity is instrumental in developing robust models capable of replicating and recognizing handwriting across different users.

5.2 Data Preprocessing

InkWave leverages a pre-processed version of the IAM Handwriting Dataset, specifically the .pt files made available through the DiffusionPen Hugging Face repository. Instead of using raw data, the handwriting samples are transformed into fixed-size grayscale strips, deskewed and centered for uniformity, normalized to a consistent height compatible with the Latent Diffusion Model (LDM), and packaged as PyTorch tensors to facilitate rapid data loading. The dataset is also methodically divided into training, validation, and test sets, residing in the saved_iam_data/ directory, eliminating the need for additional preprocessing steps before model training.

The preprocessing pipeline converts each handwritten word into a clean, uniform input for the model. All images are cropped from full IAM pages, deskewed to fix tilt, converted to normalized grayscale, and resized to a fixed height suitable for the diffusion model. Noise, shadows, and background artifacts are removed, and the final images are stored as PyTorch tensors. This ensures every sample has a consistent shape, quality, and format for fast and stable training and inference.

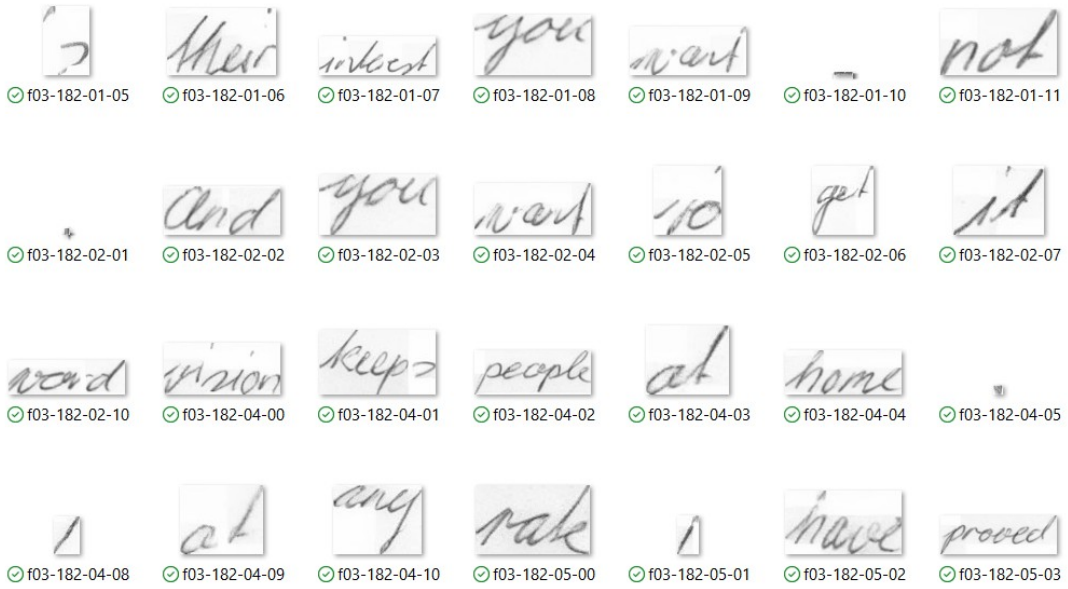


Figure 5.1: Snippet from IAM Dataset

5.3 Handwriting Detection Module

This module uses Microsoft TrOCR (Transformer-based OCR) to extract text from handwritten images. The model processes uploaded images locally, runs inference to recognize handwritten characters, and returns the detected text as a string. It supports CPU and GPU inference and is used for both user-facing text extraction and internal validation of generated handwriting quality.

5.3.1 Model Architecture

The module includes:

Vision Encoder (ViT – Vision Transformer): The Vision Transformer processes the handwritten image by dividing it into patches, converting each patch into an embedding, and applying self-attention to learn stroke patterns, shapes, and spatial relationships. This produces a sequence of visual feature vectors that represent the handwriting in a structured and meaningful way.

Text Decoder (Transformer Decoder): The Transformer decoder generates text one token at a time by attending to the encoder's visual features. Through cross-attention, it focuses on the relevant parts of the handwriting image while predicting each character, allowing it to handle variations in slant, spacing, and writing style.

Encoder–Decoder Interaction: Cross-attention serves as the bridge between the encoder and decoder. As the decoder predicts each token, it selectively attends to different encoder features, effectively "reading" the handwriting image as it generates the text. This interaction ensures high accuracy even with irregular handwriting.

Model Variant Used:

- microsoft/trocr-base-handwritten
- Pretrained on diverse handwritten text datasets

5.4 Handwriting Generation Module

Generates handwritten text images from text input using diffusion models. Uses a UNet-based diffusion model trained on the IAM dataset, with a style encoder that extracts handwriting features from reference samples. Supports custom style transfer by learning from uploaded handwriting images. Generates word-level or sentence-level outputs by processing text through the diffusion pipeline, producing realistic handwritten images that match the learned style characteristics.

5.4.1 Model Architecture

This module includes:

UNet Structure: The UNet follows a U-shaped encoder–decoder design with residual blocks and attention layers at key resolutions. The downsampling path captures local stroke details, the bottleneck integrates global context through self-attention, and the upsampling path reconstructs fine handwriting structure using skip connections. The model uses base 320 channels with increasing multipliers to capture higher-level patterns.

Conditioning Mechanisms: Generation is guided by multiple conditioning inputs. Text conditioning is applied via cross-attention using CANINE embeddings, ensuring characters are generated in the correct sequence. Style conditioning injects the processed style vector into the UNet time embeddings, influencing slant, pressure, spacing, and overall handwriting feel. The architecture also supports optional writer ID conditioning for preset author styles.

Diffusion Process: The model uses a DDIM scheduler with 50 inference steps, where the UNet gradually denoises randomly sampled latent noise into a coherent

handwriting strip. A linear noise schedule (1e-4 to 0.02) is used during training. Once denoising completes, the latent is decoded through the Stable Diffusion VAE to produce the final handwriting image consistent with both text and style conditions.

5.5 System Integration

5.5.1 Prerequisites

The prerequisites for the web app are as follows:

- Python 3.7 or higher
- CUDA-capable GPU or CPU
- PyTorch (>=2.0.0)
- FastAPI (>=0.100.0)
- Uvicorn (>=0.23.0)
- Transformers (>=4.30.0)
- Diffusers (>=0.21.0)
- Pillow (>=9.0.0)
- OpenCV (>=4.8.0)
- ReportLab (>=4.0.0)

5.5.2 Frontend Implementation

The frontend of the AI Based Handwriting Generation system features a user-friendly interface designed to simplify interaction with the handwriting generation. It includes clear input fields for text entry, intuitive buttons for uploading files, and a visually appealing layout that guides users seamlessly through the process of generating and downloading handwriting samples. Clarity and accessibility are given attention, ensuring that users can easily navigate the interface and access all functionalities without unnecessary complexity.

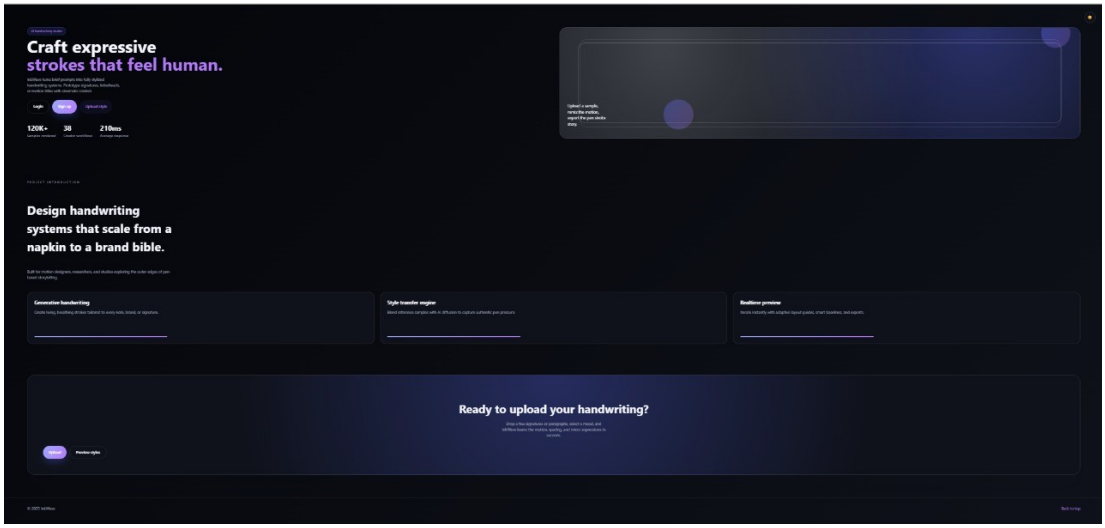


Figure 5.2: InkWave Landing Page

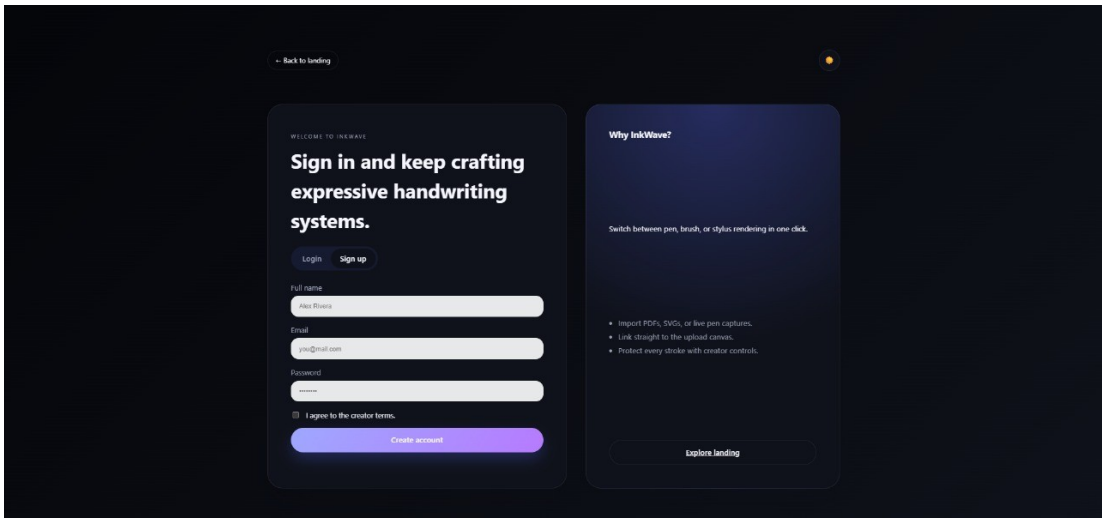


Figure 5.3: InkWave Sign Up Page

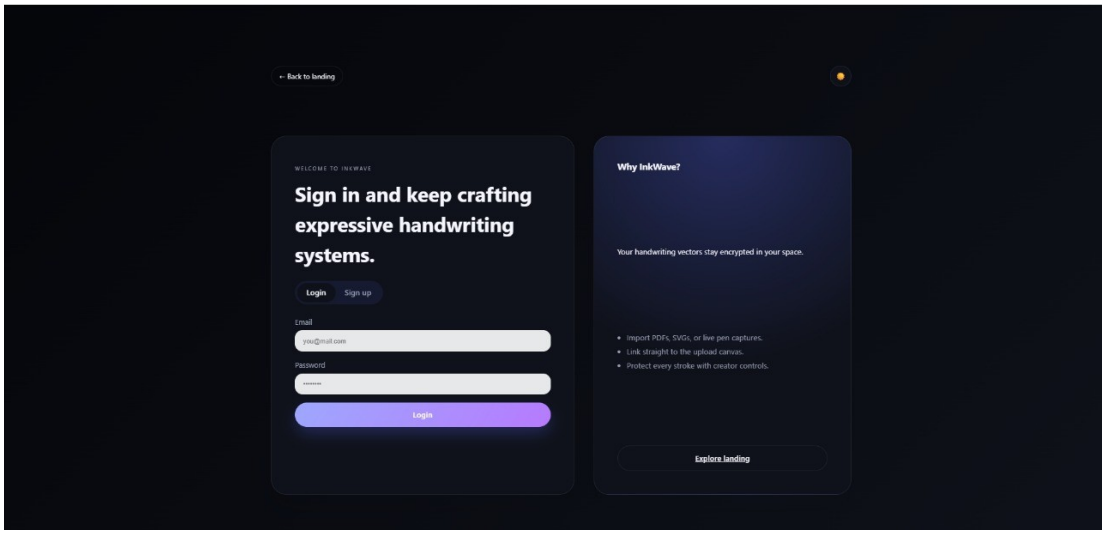


Figure 5.4: InkWave Login Page

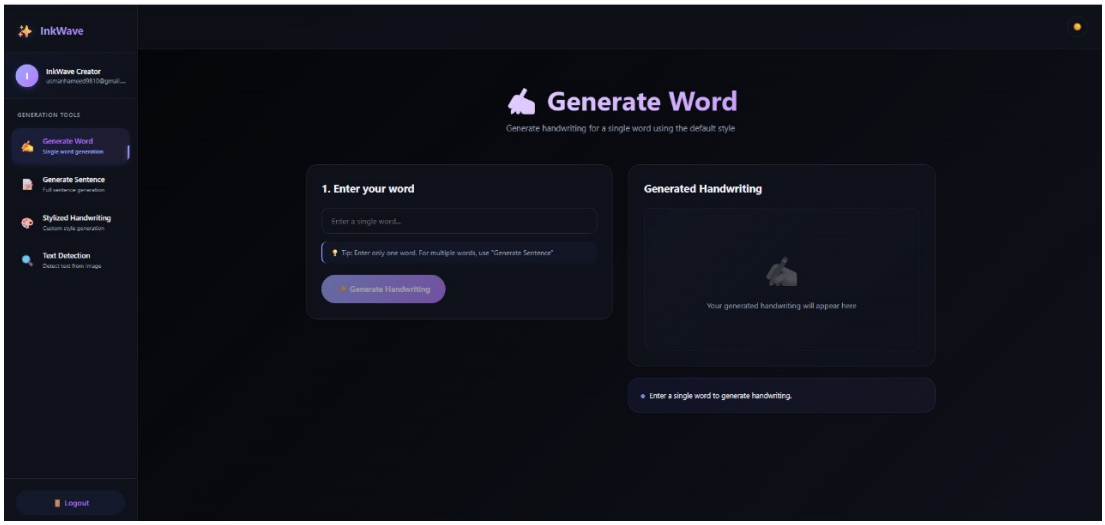


Figure 5.5: InkWave Generate Word Page

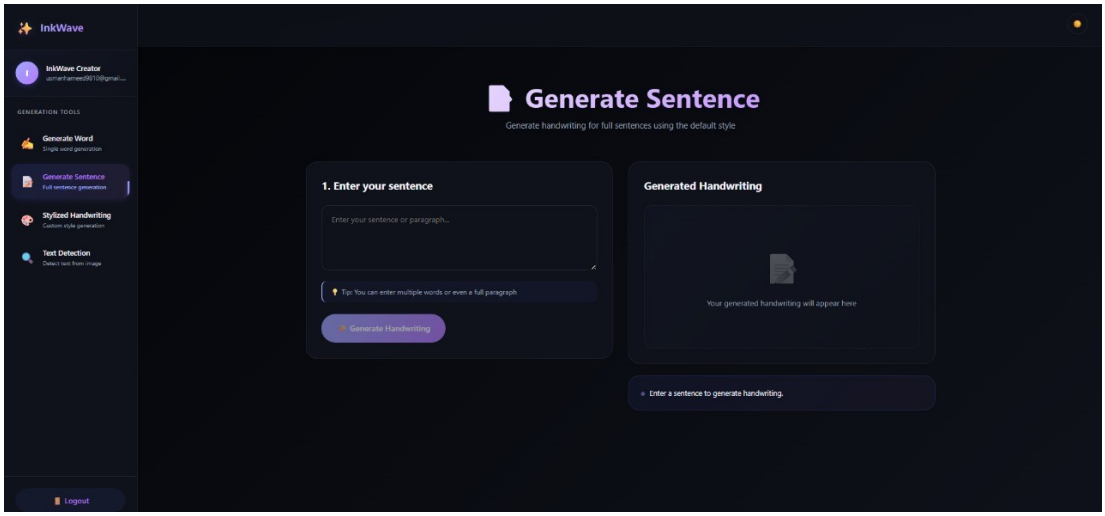


Figure 5.6: InkWave Generate Sentence Page



Figure 5.7: InkWave Stylized Handwriting Generation Page

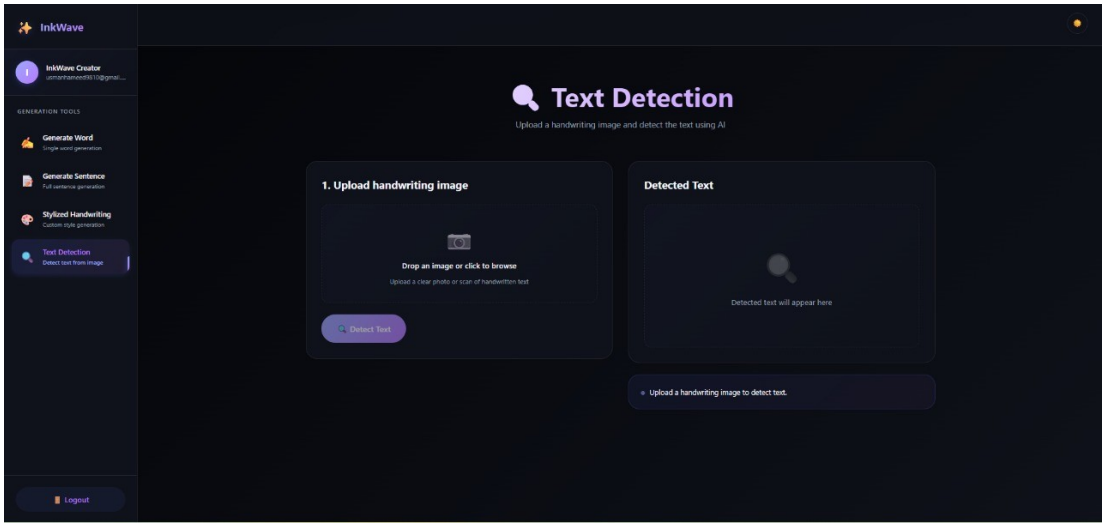


Figure 5.8: Handwriting Detection Page

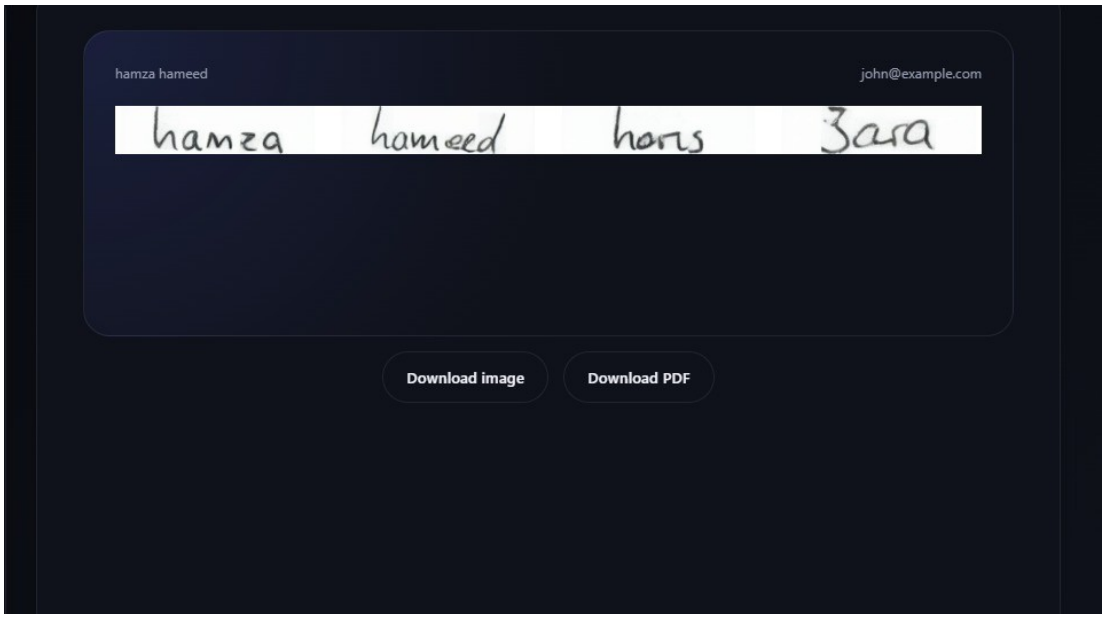


Figure 5.9: Preview Output Page

5.5.3 Backend Implementation

The backend of AI Based Handwriting Replication App is built using FastAPI, functioning as a REST API that manages handwriting generation, style conditioning, and text detection. It integrates a conditional diffusion model where a UNet denoiser, trained on preprocessed IAM word images, operates in latent space using a Stable Diffusion VAE. Text inputs are encoded using CANINE, a character-aware transformer that produces 320-dimensional embeddings, while handwriting style is extracted through a MobileNet/ResNet-based encoder and injected into the diffusion process via time-step embeddings. The API exposes three generation routes: the word

endpoint, which generates single-word handwriting through DDIM sampling over 50 denoising steps; the sentence endpoint, which splits input text into words, synthesizes each word separately, and stitches the results with horizontal padding; and the stylized endpoint, which accepts multiple reference samples, extracts their style features, and conditions the UNet to produce handwriting matching the user's style. All model inference runs through subprocess-driven Python scripts that handle tokenization, conditioning, iterative denoising, latent decoding, and output image creation. A built-in HTR service powered by Microsoft TrOCR (ViT encoder + Transformer decoder) provides handwriting recognition for internal validation and detection requests. The backend supports multipart uploads, saves user samples in UUID-based directories, returns outputs as base64 images, and includes a PDF generator that embeds handwriting images into PDF documents. Optional authentication uses MongoDB for user accounts, bcrypt for password hashing, and JWT for secure session management. Running on Uvicorn with CORS enabled and async handling, the backend cleanly separates API routing from model inference, allowing the deep-learning components and web server to be scaled or maintained independently.

```python
def preprocess_custom_image(img_path, transform, args):
    """Preprocess custom handwriting images to match model expectations"""
    try:
        # Load image
        img = Image.open(img_path).convert('RGB')

        # Resize to 64px height while maintaining aspect ratio
        (original_width, original_height) = img.size
        new_width = int(original_width * 64 / original_height)
        img_resized = img.resize((new_width, 64))

        # Pad width to 256px
        if new_width < 256:
            img_padded = ImageOps.pad(img_resized, size=(256, 64), color="white")
        else:
            # If too wide, gradually resize down
            current_width = new_width
            current_img = img_resized
            while current_width > 256:
                current_width = max(256, current_width - 20)
                current_img = current_img.resize((current_width, 64))
            img_padded = ImageOps.pad(current_img, size=(256, 64), color="white")

        # Convert to tensor and normalize
        img_tensor = transform(img_padded).to(args.device)
        return img_tensor

    except Exception as e:
        print(f"Error processing {img_path}: {e}")
        return None
```

Figure 5.10: Custom Preprocessing of Handwriting Images for Model Compatibility

```python
def preprocess(path, transform):
    img = Image.open(path).convert("RGB")
    w, h = img.size
    new_w = int(w * 64 / h)
    img = img.resize((new_w, 64), Image.LANCZOS)
    if new_w > 256:
        img = img.resize((256, 64), Image.LANCZOS)
    if new_w < 256:
        img = ImageOps.pad(img, (256, 64), color="white")
    return transform(img)


def collect_reference_images(style_dir, transform, device, limit):
    patterns = ("*.png", "*.jpg", "*.jpeg", "*.bmp")
    paths = []
    for pattern in patterns:
        paths.extend(glob.glob(os.path.join(style_dir, pattern)))
    paths = [p for p in paths if os.path.isfile(p)]
    if not paths:
        raise ValueError(f"No reference images found inside {style_dir}")
    print(f"Found {len(paths)} reference images, using up to {limit}")
    tensors = [preprocess(p, transform).to(device) for p in paths[:limit]]
    return torch.stack(tensors)
```

Figure 5.11: Resizing and Padding Handwriting Samples

```python
class PatchedUNetModel(UNetModel):
    """Patched UNet that can handle custom style features"""

    def forward(self, x, timesteps=None, context=None, y=None, original_images=None, mix_rate=None, style_extractor=None, **kwargs):
        """
        Forward pass with support for custom style features
        """
        # If we have custom style features, use them directly
        if style_extractor is not None and original_images is None:
            print(f"Using custom style features with shape: {style_extractor.shape}")
            # Store the custom style features for use in the UNet
            self.custom_style_features = style_extractor
            # Call parent forward but don't pass original_images to avoid the style extraction pipeline
            return super().forward(x, timesteps, context, y, **kwargs)
        else:
            # Use the original forward pass
            return super().forward(x, timesteps, context, y, original_images=original_images, mix_rate=mix_rate, style_extractor=style_extractor, **kwargs)
```

Figure 5.12: Handling Custom Style Features in Handwriting Generation

Figure 5.13: Extraction Style Features

```python
def _combine_images(image_paths: List[Path], output_path: Path, padding: int = 8) -> Path:
    images = [Image.open(p).convert("RGB") for p in image_paths]
    target_height = max(img.height for img in images)
    resized = []
    for img in images:
        if img.height != target_height:
            new_img = Image.new("RGB", (img.width, target_height), "white")
            offset = (target_height - img.height) // 2
            new_img.paste(img, (0, offset))
            resized.append(new_img)
        else:
            resized.append(img)
    total_width = sum(img.width for img in resized) + padding * (len(resized) - 1 if len(resized) > 1 else 0)
    canvas = Image.new("RGB", (total_width, target_height), "white")
    cursor = 0
    for img in resized:
        canvas.paste(img, (cursor, 0))
        cursor += img.width + padding
    output_path.parent.mkdir(parents=True, exist_ok=True)
    canvas.save(output_path)
    return output_path
```

Figure 5.14: Combining Multiple Handwriting Word Images into a Full Sentence Image

5.5.4 Backend API Call

```python
@app.post("/api/word")
async def generate_word(payload: TextPayload):
    text = _ensure_text(payload.text)
    image_path = await asyncio.to_thread(_generate_word_image, text)
    return {
        "text": text,
        "filename": image_path.name,
        "image_base64": _encode_image(image_path),
    }


@app.post("/api/sentence")
async def generate_sentence(payload: TextPayload):
    text = _ensure_text(payload.text)
    words = _split_words(text)
    word_images = []
    for word in words:
        image_path = await asyncio.to_thread(_generate_word_image, word)
        word_images.append(image_path)
    merged_path = API_OUTPUT_DIR / f"sentence_{uuid.uuid4().hex}.png"
    merged_path = await asyncio.to_thread(_combine_images, word_images, merged_path)
    return {
        "text": text,
        "word_count": len(words),
        "filename": merged_path.name,
        "image_base64": _encode_image(merged_path),
    }


@app.post("/api/stylized")
async def generate_stylized(
    text: str = Form(..., description="Text to render using the uploaded handwriting samples"),
    samples: List[UploadFile] = File(..., description="At least five handwriting reference images"),
):
    clean_text = _ensure_text(text)
    if len(samples) < 5:
        raise HTTPException(status_code=400, detail="Please upload at least five reference images.")

    style_set_id = uuid.uuid4().hex
```

Figure 5.15: Main API POST Call

This defines a web API using FastAPI for AI Based Handwriting Generation and Handwritten Text Recognition (HTR). The following are key endpoints:

- **/api/word (POST)**

  Accepts a single word in JSON, forwards it to the handwriting generation service

using pre-trained IAM styles, runs diffusion in latent space via Stable Diffusion VAE and DDIM (50 steps), and returns a base64 handwritten image with metadata. Non-blocking internal HTR validation is also performed.

- **/api/sentence (POST)**
  Takes a full sentence, splits it into words, generates each word individually using the word generator, merges the outputs horizontally with spacing, and returns the combined sentence image as base64 along with word-count metadata.
- **/api/stylized (POST)**
  Receives text plus at least five reference handwriting samples via multipart form-data. The server saves the samples, extracts style features using the style encoder, conditions the diffusion model with these features, generates each word in the user's style, merges them, and returns the final stylized image in base64 form with the stored reference paths.
- **/api/detect (POST)**
  Accepts an uploaded handwritten image and uses the TrOCR-based HTRService to extract text. Returns the detected string as JSON. Used for handwriting recognition and internal quality checks.
- **/convert/pdf (POST)**
  Accepts a generated handwriting image and the original text, inserts the image into a centered, properly scaled A4 PDF, adds footer metadata, and returns the PDF as a downloadable file response.
- **/auth/signup (POST)**
  Receives name, email, and password; creates a new user in MongoDB; hashes the password with bcrypt; and returns a JWT token with user information.
- **/auth/login (POST)**
  Validates email and password, authenticates the user against the database, and returns a JWT token and user data on success.
- **/auth/me (GET)**
  Returns the authenticated user's information using the JWT token provided in the Authorization header.

Key Features:

- Uses dependency injection (Depends) to load HTR service and authentication dependencies.

- Validates input data (e.g., checks for empty text, minimum file requirements, authentication tokens).
- Returns appropriate HTTP error responses for invalid or failed requests.
- Implements lazy loading for ML models to optimize memory usage.
- Implements singleton pattern for service instances to avoid redundant model loading.
- Handles temporary file cleanup automatically after processing.
- Supports both pre-trained styles and custom user-uploaded styles.
- Converts generated images to PDF format with proper formatting and metadata.

5.6 Chapter Summary

This chapter detailed the complete design and implementation of AI Based Handwriting Replication App, covering the dataset foundation, preprocessing pipeline, model architectures, and full system integration. It explained how handwriting detection and generation models work together, supported by CANINE text embeddings, style extraction, and diffusion-based synthesis, to produce realistic handwritten outputs. The chapter also outlined how the FastAPI backend, frontend interface, and authentication system connect seamlessly to deliver a smooth user experience. Altogether, this chapter demonstrates how each component contributes to a unified, efficient, and intelligent handwriting generation platform.

CHAPTER 6

SYSTEM TESTING AND EVALUATION

This chapter describes how the handwriting generation system was tested and evaluated. Since the project consists of several connected components, such as the frontend interface, backend API, handwriting sample processing, and output generation, each part was tested individually and then combined for full system testing. The goal was to ensure that the system works smoothly, produces accurate handwriting output, and provides a simple user experience.

6.1 Testing Methodology

The testing strategy was divided into three main stages:

- Module Testing: Each component such as text input, file upload, sample validation, handwriting output, and PDF/image download was tested separately.
- Integration Testing: After verifying module-level functionality, all components were connected and tested together to ensure the system works end-to-end.
- Performance Testing: The system was tested with different handwriting words and samples, using different devices and browsers to evaluate ease of use, speed, and stability.

6.2 Functional Testing

6.2.1 Landing Page Testing

The landing page displays the introductory content, project description, and upload prompt. Testing showed that:

- The page loads quickly.
- Buttons and navigation links work properly.
- UI animations and hover effects behave correctly.

6.2.2 Login & Sign-Up Page Testing

Both authentication pages were tested for:

- Account creation
- Login validation
- Incorrect password warnings
- Navigation back to the landing page

All forms responded correctly to both valid and invalid inputs.

6.2.3 Handwriting Generator Page Testing

This is the main page where users type text, upload handwriting samples, and generate their custom output.

Testing confirmed:

- Text box accepts short and long input
- System prevents users from generating handwriting with fewer than five samples
- Upload buttons and drag-and-drop both work
- “Convert to handwriting” activates only when requirements are met
- Output preview updates automatically after generation

6.2.4 Preview & Download Testing

Users can download the generated handwriting as:

- PNG image
- PDF document

Both download options worked correctly during testing.

6.3 Handwriting Detection Module Evaluation

The Handwriting Detection Module was tested across multiple scenarios:

Detection Accuracy

- Performs strongly on clean handwriting samples
- Moderately affected by tilt or rotation

- Model detects content but not stylistic elements (expected behavior)

Boundary Box Precision

- Bounding boxes were generated accurately in most cases
- Slight deviations occurred with noisy backgrounds

Error Handling

- Corrupted files rejected safely
- Non-image formats blocked

Evaluation Summary

Overall, the module is accurate, stable, and suitable for student-level and research-level application requirements.

6.4 Handwriting Generation Module Evaluation

The Handwriting Generation Module was evaluated to determine its accuracy, realism, and overall reliability across different text lengths and sample qualities.

Style Accuracy

- Generated handwriting closely matched the user's stroke patterns
- Slant and curvature were replicated consistently
- Character identity remained faithful to the uploaded samples
- Accuracy for short and medium-length inputs
- Stylistic drifts observed in paragraph generations

Output Quality & Realism

- Single-word generation produced highly clear and visually consistent results
- Sentence-level outputs preserved spacing and alignment effectively
- Output realism improved significantly when high-quality samples were provided
- Generated handwriting maintained natural flow without looking artificially repeated

Performance

- Fastest generation times on GPU (4 minutes)
- Model remained stable across repeated generation tests
- Memory usage spiked during diffusion inference but stayed within acceptable limits

Evaluation Summary

Overall, the Handwriting Generation Module produced style-accurate handwriting, especially for short and medium-length text. The system remained stable and consistent throughout testing, with acceptable performance for academic and prototype use.

6.5 System Usability & Performance Testing

6.5.1 Browser Testing

The system was tested on different browsers which were Google Chrome, Microsoft Edge, and Opera. The fastest response time was on Google Chrome Browser, loading within five seconds The slowest load time was on the Microsoft Edge browser which at least took 15 seconds. This concludes that web app is optimized to work well on modern browsers.

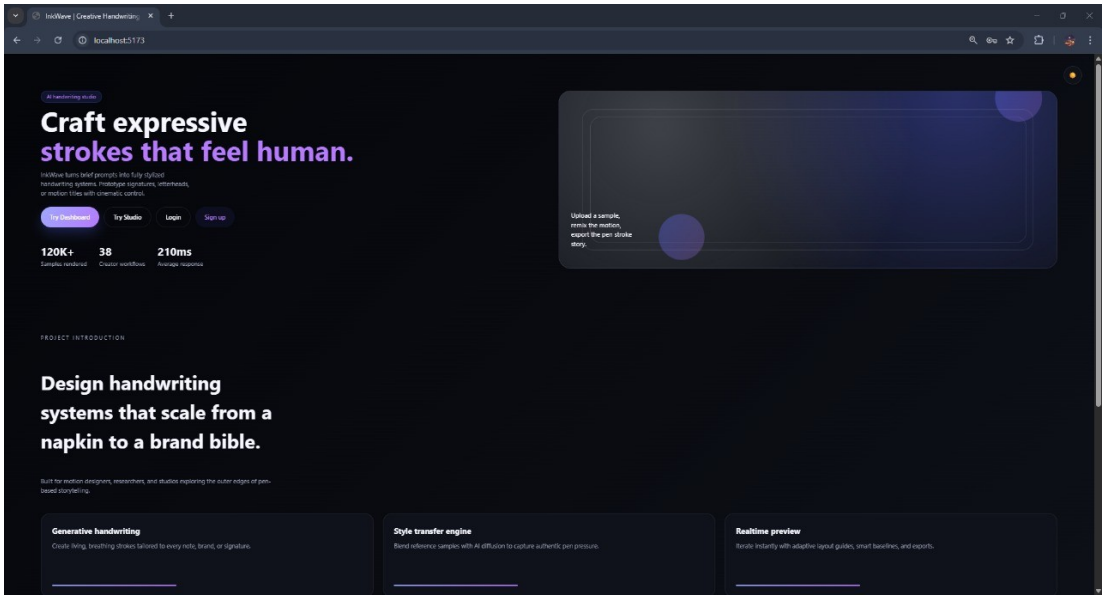


Figure 6.1: Web app compatibility on Chrome browser

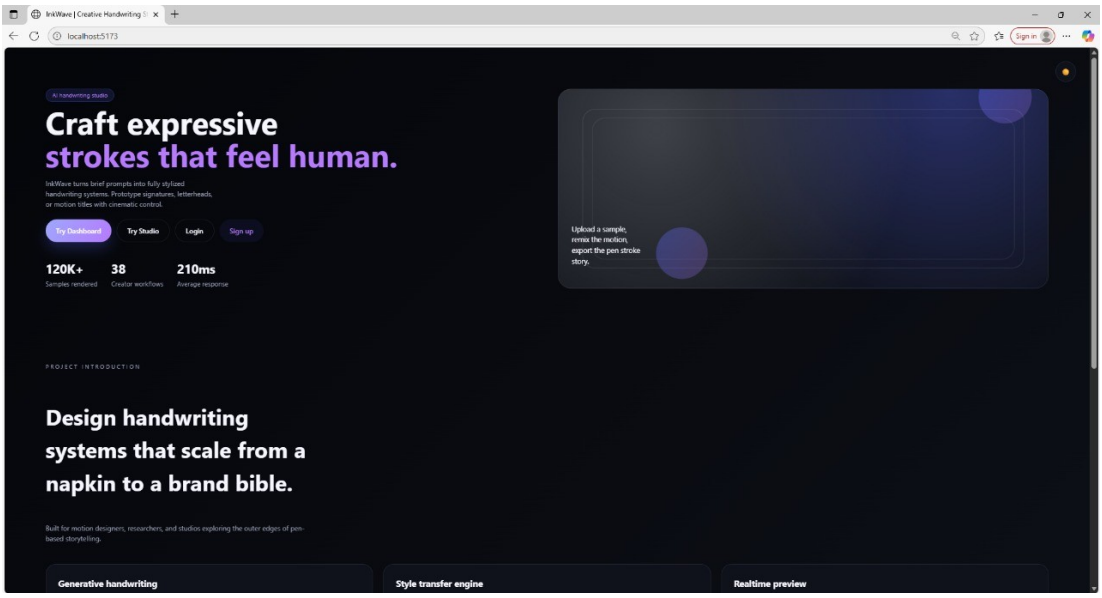


Figure 6.2: Web app compatibility on Microsoft Edge browser

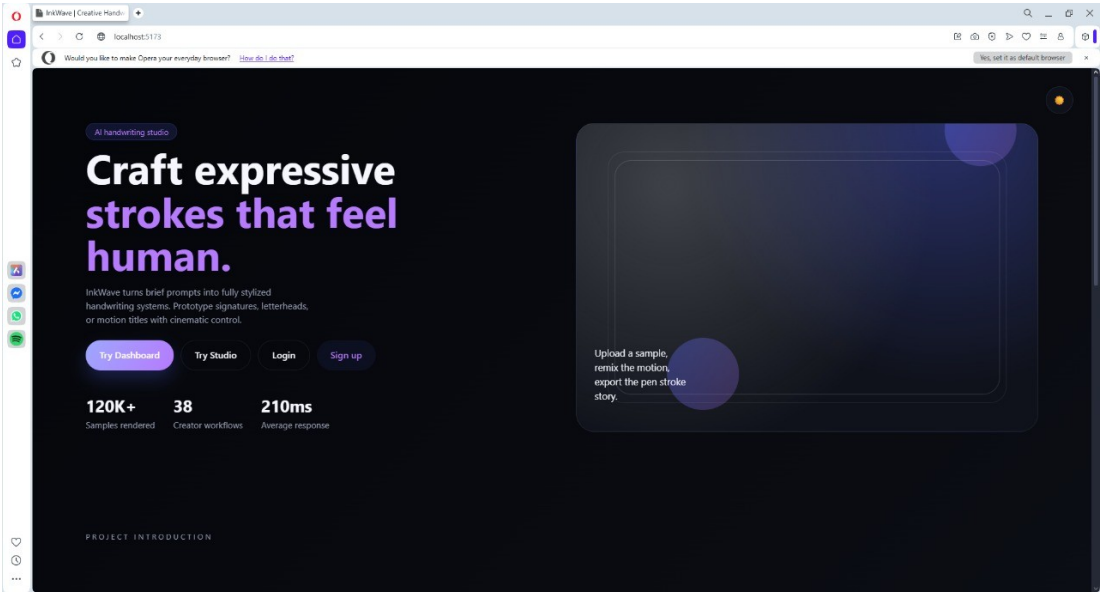


Figure 6.3: Web app compatibility on Opera browser

6.5.2 User Testing

It was discovered in user testing that the interface is contemporary, clean, and user-friendly, as there is no issue in moving around the various features. The dark theme increased the readability of the text and made the uploaded handwriting samples easier to see. User found the uploading experience to be easy, and both drag-and-drop and manual selection worked well. Live preview was also a significant convenience that was emphasized and allowed the user to be confident in what is being generated before downloading it. Some of the users suggested that guidance on the quality of a good

handwriting sample should be included to assist in achieving the best outcomes. In general, the system was rated as having high usability performance as the testers deemed the application as being easy, responsive, and user-friendly.

6.5.3 Test Case

Table 6.1 Test Cases

ID	Test Case	Expected Result	Actual Result	Status
1	Load Landing Page	Load < 7 sec	4-6 sec	PASS
2	Load app on Chrome	App loads within 10 seconds	App loaded in under 5 seconds	PASS
3	Load app on Microsoft Edge	App loads within 10 seconds	App loaded in under 5 seconds	PASS
4	Load app on Opera	App loads within 10 seconds	App loaded in under 5 seconds	PASS
5	Upload ≥5 samples	Show previews	Works	PASS
6	Upload <5 samples	Warning shown	Works	PASS
7	Generate Word	Output handwriting	Works	PASS
8	Generate Sentence	Output handwriting	Works	PASS
9	Stylized Generation	Style-based handwriting output	Works	PASS
10	Detect Text	HTR extraction	Works correctly	PASS
11	Download as Image	File downloaded	Works correctly	PASS
12	Download as PDF	PDF exported	Works correctly	PASS

6.6 API Testing

API testing was carried out using both FastAPI's interactive documentation and Postman to verify the correctness, performance, and stability of all backend endpoints. The main endpoints tested included **api/word**, **api/sentence**, **api/stylised**, and **api/pdf**.

- FastAPI's /docs interface was used to test request/response behavior and validate JSON schemas.
- Postman was used for deeper testing, including negative cases, repeated

requests, and timed performance checks.

- The api/word and api/sentence endpoints were tested for text input handling and handwriting generation output.
- The api/stylised endpoint was tested for custom style extraction using multiple handwriting samples.
- The api/pdf endpoint was verified for converting generated handwriting into PDF format without errors.
- Response times were optimized by improving preprocessing and using asynchronous FastAPI routes.
- All APIs returned correct status codes, handled invalid inputs safely, and showed stable performance.

6.7 Chapter Summary

The entire testing and evaluation of the handwriting replication system was done in this chapter. Structured testing with FastAPI, Postman and real user interaction was used to test all the components and ensure that the frontend workflows, backend APIs, and generation modules worked as intended. Findings indicated that the system is reliable, has correct handwriting outputs, and graciously copes with errors. In general, the app achieves the desired purposes and is highly stable to be used in academia and prototype applications.

CHAPTER 7

CONCLUSION

This report described the creation of an artificial intelligence-related handwriting replication system that will enable users to enter several samples of the handwriting and create new text in a similar style. During the project, multiple methods were researched to get handwriting features, user input and produce real handwriting. The system can learn writing style of the user and generate new text in handwritten form which is very similar to the samples.

The entire process was constructed as a workflow, beginning with the upload of the image, then the style learning and text generation followed by a clean and user-friendly interface that shows the generated handwriting. The system was combined with a lightweight backend and was tested on sample data as well as on actual user input. In general, the project shows that the AI-based handwriting generation can be conducted and can be efficient, providing an original and individualized means of the users to produce the custom handwritten texts.

7.1 Key Takeaways

In conclusion, this report shows the design and implementation of a successful design in generating personalized handwriting. The project demonstrates that artificial intelligence can acquire the patterns of writing based on a limited number of examples and reproduce the style in a natural and visually persuasive way. It is user friendly and capable of creating handwritings to the typed custom texts and is applicable in creative writing, personalized notes, electronic signatures and other applications.

7.2 Limitations and Challenges

Even though the system is working well in general, there were some issues encountered throughout the development. The reliance on the quality of the handwriting samples submitted by the users is one of the greatest limitations; blurred, faint, or inconsistent handwriting samples decrease the accuracy of the produced handwriting. The other problem is the creation of longer text since the system is still more effective with short sentences or lines and long paragraphs can lead to spacing or alignment problems. Moreover, the processing time might be long on the computers that lack powerful hardware support since the process of handwriting generation involves several steps before completion. The support to other languages is also not so

much supported now because the system was trained and tested with English handwriting samples. The generation of handwritings would consume a lot of processing power particularly when incurring inferences and therefore, a large number of users would lead to delays or slow down the system. Lastly, the system does not simulate the movement of the pen by replicating its motion, instead producing fixed handwriting images, and this inhibits the capability of the system to replicate the flow of a natural handwriting in a dynamic form. Such issues outline the aspects of future enhancement and development.

7.3 Future Developments

There are a handful of ways in which the system can be enhanced and expanded in a meaningful manner. The developments that would improve usability, performance, and scalability would be as follows.

- **Multi-Language and Script Support:** The trained system can be adapted to a more diverse and broader user base by training on datasets of other languages like Urdu, Arabic, Chinese or regional scripts.
- **Real-Time and Faster Handwriting Generation:** The model could be optimized, or more efficient architectures could be used to generate results at a faster rate. Live handwriting generation can be of great benefit to the user experience, particularly in cases of mobile users.
- **Full Paragraph and Page-Level Handwriting:** Future versions may feature the ability to handle layouts in a better way to create long paragraphs, multiple lines and entire handwritten pages in a way that does not misalign or have spacing problems.
- **Creation of a Mobile Application:** It would be more convenient to have a mobile version of the system, so that people who need quick handwritten notes or customized writing on-the-fly can have the possibility to do it.
- **Style Adjustment by the user:** It would be more user-controlled by allowing the user to adjust the style settings to their preference like slant, spacing, thickness or neatness.

- **Note-taking and Document Integrations:** The system can be integrated with popular applications such as Notion, OneNote, Google Docs or PDF editors so that users can directly create handwritten notes in those applications.

- **Security and Watermarking Features:** To curb misuse, it may also include invisible watermarks or verification markers to generated handwriting, making it impossible to use it to commit forgery or other unauthorized actions.

- **Scalable Server Deployment:** The cloud-based GPU servers or distributed processing would enable the system to serve a large number of users simultaneously, eliminating the existing constraint due to resource availability.

7.4 Chapter Summary

Conclusively, the project presents a working AI generated handwriting generation system which trains and learns through a collection of handwritten samples by a user and generates realistic handwritten text. Although it has some limitations regarding the use of resources, language support, and long-text generation, the system offers a large base to further development and presents a plethora of creative and valuable applications.

REFERENCES

[1] J. Gan and W. Wang, “HiGAN: Handwriting imitation conditioned on arbitrary-length texts and disentangled styles,”
in Proc. AAAI-21, 2021, pp. 7484–7492.
Available: https://ojs.aaai.org/index.php/AAAI/article/view/16917

[2] K. Nikolaidou, G. Retsinas, G. Sfikas and M. Liwicki, “DiffusionPen: Towards Controlling the Style of Handwritten Text Generation,”
in Computer Vision – ECCV 2024, Springer, 2024.
Available: https://link.springer.com/chapter/10.1007/978-3-031-73013-9_24#citeas

[3] kimari, muchigi; Fry, Edward; Nwaogu, Ikenna; Bennett, YuMei; and Santerre, John (2021) "Generating and Smoothing Handwriting with Long Short-Term Memory Networks," SMU Data Science Review: Vol. 5: No. 1, Article 4.
Available at: https://scholar.smu.edu/datasciencereview/vol5/iss1/4

[4] H. A. Alhamad et al., “Handwritten Recognition Techniques: A Comprehensive Review,”
Symmetry, vol. 16, no. 6, art. 681, 2024.
Available: https://doi.org/10.3390/sym16060681

[5] S. H. Ali and M. B. Abdulrazzaq,
“A Comprehensive Overview of Handwritten Recognition Techniques: A Survey,”
J. Comput. Sci., vol. 19, no. 5, pp. 569–587, 2023.
Available: https://doi.org/10.3844/jcssp.2023.569.587

[6] M. Li et al., “TrOCR: Transformer-based Optical Character Recognition with Pre-trained Models,”
arXiv preprint arXiv:2109.10282, 2021.
Available: https://arxiv.org/abs/2109.10282

[7] S. Ferro, A. Torcinovich, A. Traviglia, and M. Pelillo, "Boosting CNN-based Handwriting Recognition Systems with Learnable Relaxation Labelling," *Pattern Recognition Letters*, vol. 187, pp. 200–207, 2024.
https://arxiv.org/pdf/2409.05699

[8] J. Wang, L. Mou, C. Zheng and W. Gao, "Image-Based Freeform Handwriting Authentication with Energy-Oriented Self-Supervised Learning," in *IEEE Transactions on Multimedia*, vol. 27, pp. 1397-1409, 2025
https://ieeexplore.ieee.org/abstract/document/10812847

[9] O. Yakovchuk and M. Vasin, “Increasing the accuracy of handwriting text recognition in medical prescriptions with generative artificial intelligence”, *TAPR*, vol. 4, no. 2(72), pp. 18–21, Aug. 2023.
Available: https://journals.uran.ua/tarp/article/view/284998

[10] S. Ahlawat, A. Choudhary, A. Nayyar, S. Singh, and B. Yoon,
“Improved handwritten digit recognition using convolutional neural networks (CNN),” Sensors, vol. 20, no. 12, p. 3344, 2020.
Available: https://doi.org/10.3390/s20123344

[11] “Handwriting recognition using artificial intelligence neural network and image processing,” International Journal of Advanced Computer Science and Applications (IJACSA), vol. 11, no. 7, 2020.
Available: https://dx.doi.org/10.14569/IJACSA.2020.0110719

[12] M. Díaz, A. Mendoza-García, M. A. Ferrer, and R. Sabourin, “A survey of handwriting synthesis from 2019 to 2024: A comprehensive review,” Pattern Recognition, vol. 162, art. 111357, 2025.
Available: https://doi.org/10.1016/j.patcog.2025.111357

[13] R. R. P. Roshan, S. K. Swetha, and V. Mathew, “An overview of handwritten text recognition using generative AI: Cutting-edge approaches, key challenges, and future directions,” International Journal for Research in Applied Science & Engineering Technology (IJRASET), vol. 12, no. 9, pp. 1817–1825, 2024.

Available: https://doi.org/10.22214/ijraset.2024.64315

[14] Y. Luo, C. Fang, Y. Lin, and H. Huang, “CalliRewrite: Recovering handwriting behaviors from calligraphy images without supervision,”
in Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR), 2024, pp. 16246–16255.
Available: CalliRewrite

[15] X. Huang and Y. Xiong, “A cost-effective robotic handwriting system with AI integration,” IEEE Access, vol. 13, pp. 108543–108556, 2025.
Available: https://ieeexplore.ieee.org/document/10609164

[16] J. Prem Kumar, “Handwritten text recognition using deep learning: A CNN–LSTM approach,” International Journal of Intelligent Systems and Applications in Engineering (IJISAE), vol. 13, no. 1, pp. 14–22, 2025.
Available: https://ijisae.org/index.php/IJISAE/article/view/7443

[17] S. Wang, R. Wang, M. A. Ferrer, and R. Sabourin, “Image-based freeform handwriting authentication with energy-oriented self-supervised learning,”
IEEE Transactions on Multimedia, vol. 26, pp. 8143–8157, 2024.
Available: https://ieeexplore.ieee.org/document/10411425

[18] L. Kang, P. Riba, M. Rusiñol, A. Fornés, and M. Villegas, “GANwriting: Content-conditioned generation of styled handwriting,” in Proc. 16th European Conf. on Computer Vision (ECCV), 2020, pp. 273–289.
Available: https://doi.org/10.1007/978-3-030-58558-7_17

[19] H. Dai, X. Liu, J. Li, and Z. Li, “One-DM: One-shot diffusion mimicker for handwritten text generation,” in Computer Vision – ECCV 2024, Springer, 2025.
Available: https://www.arxiv.org/abs/2407.04737